

B. In your group, discuss the following issues with general descriptions and specific examples (without referring to the lecture slides):

1. What are the types of business drivers/goals for software systems? In general, what is the point of having software systems? Who sets the business goals? Are all business goals consistent and equal?

=> There are several types of business drives for software development including retaining or increasing customers, improving customer relationships, increasing productivity and efficiency, replacing outdated technology, responding to legal or social changes, and so on.

2. What are the types of requirements for a software system?

=> There are three main requirements:

- **Functional Requirements:**

What the system and user accomplish together. They describe inputs, outputs, and their relationships. The IEEE text notes they should focus on **joint tasks** rather than forcing a rigid split of labor on the computer alone.

- **Data Requirements:**

What information the software must store. While functions cover actions, data requirements provide a **secure place to save records** (like names or payment history) so nothing is lost when power is off.

- **Quality Requirements (Non-Functional):**

How well and fast the system performs. They are not specific features, but rather **performance standards** that ensure the software runs smoothly and does not crash when thousands of people use it.

What are the types of non-functional requirements?

=> There are three non-functional requirements:

- **Performance & Response Time:**

This part is all about **speed and responsiveness**. It sets strict rules on how many seconds a user waits after clicking a button, preventing people from **getting stuck during busy working hours**.

- **Usability:**

This part ensures the software is **easy to read and practical** for a real working environment. It focuses on keeping screens simple and **flexible enough to handle real-world chaos** and interruptions.

- **Reliability & Accuracy:**

This ensures the software is **dependable and handles mistakes gracefully**. It guarantees **no unexpected glitches or critical failures** during rush hours, keeping all operational data precise and secure.

3. What is software design? What is considered in software design?

=> Software design is the process of planning and defining how a software system will work before coding begins. It bridges the gap between “what the system should do” and “how the system will do it.”

=> There are four pillars should be considered in software design including:

- + Functional => What the system does? (e.g, Transfer Money)
- + Non-functional => How well the system performs

- + Data Design: How data is stored and connect
- + UI/UX: How easy and pleasant is it for users?

4. What is the role of “model” in software design?

Simplifying Complex Systems: A model acts as a **blueprint or map** to help developers understand a system. It strips away messy details so the team can focus purely on **how parts connect**.

Postponing Early Design Splits: Models like task descriptions show what the **human and computer accomplish together**. This prevents making premature technical decisions before understanding the business.

Bridge for Communication: It serves as a **shared language** between tech teams and clients. It allows non-technical users to easily check if the design **matches their daily work**.

What are the differences between model and reality? Why do we use models?

Model vs. Reality: Reality is full of **chaos, interruptions, and endless details** (like a crowded hotel desk). A model is a **clean, simplified snapshot** that focuses only on core steps and critical paths.

Why We Use Models: Building straight from raw reality is **too risky, expensive, and messy**. Models let you brainstorm, catch missing pieces, and **test solutions on paper** before writing code.

5. What are the main drivers for the structural design of a software system?

=> There are three main drivers for the structural design of a software system including Quality Attributes, Primary Functionality, and Constraints

6. Any questions from the weekly submissions?

=> What types of architecture are best and when should I apply each of them?